

Contenidos vistos: Concurrency en Java

1. Fundamentos de Hilos y Concurrency

- **Concepto de Hilo (Thread):** Unidad mínima de ejecución dentro de un proceso. Permite que una aplicación realice múltiples tareas simultáneamente en arquitecturas multinúcleo.
- **Creación de Hilos en Java:**
 - Uso de la clase `Thread` y la interfaz funcional `Runnable`.
 - Inicio de ejecución mediante el método `.start()` (nunca llamando directamente a `.run()`, ya que este último se ejecuta en el hilo principal).
- **Control del Ciclo de Vida del Hilo:**
 - **Interrupción** (`interrupt()`): Mecanismo para solicitar educadamente a un hilo que detenga su ejecución, manejando excepciones como `InterruptedException`.
 - **Sincronización de finalización** (`join()`): Permite que un hilo espere a que otro termine antes de continuar.

2. Sincronización y Exclusión Mutua

- **Condiciones de Carrera (Race Conditions):** Ocurren cuando múltiples hilos intentan modificar el mismo recurso compartido de forma simultánea, generando inconsistencias de datos.
- **Sección Crítica y Monitores:**
 - **Bloques y Métodos `synchronized`:** Utilizan el cerrojo intrínseco (*intrinsic lock* o *monitor*) de un objeto para asegurar que solo un hilo a la vez acceda a la sección crítica.
 - **Locks Explícitos (`ReentrantLock`):** Alternativa avanzada a `synchronized` que ofrece mayor control, cómo intentar adquirir el cerrojo con un tiempo de espera (`tryLock()`).

- **Lectura/Escritura (ReadWriteLock):** Optimización para escenarios donde las lecturas son frecuentes y las escrituras raras, permitiendo lecturas concurrentes.

3. Visibilidad y Colecciones Concurrentes

- **Memoria de Hilos vs. Memoria Principal:**
 - **Uso de volatile:** Garantiza la visibilidad de cambios en variables entre distintos hilos, evitando que las lecturas/escrituras sean consolidadas únicamente en la caché local del núcleo de la CPU.
- **Estructuras de Datos Atómicas y Concurrentes:**
 - Clases del paquete `java.util.concurrent.atomic` (ej. `AtomicInteger`, `AtomicReference`) para operaciones *lock-free* utilizando instrucciones de CPU tipo CAS (Compare-And-Swap).
 - Colecciones seguras para hilos como `ConcurrentHashMap` y colecciones de bloqueo como `BlockingQueue`.

4. Gestión de Hilos con Framework Executor

- **Interfaces Executor y ExecutorService:** Abstracción para desacoplar el envío de tareas de su ejecución.
- **Pools de Hilos (ThreadPools):**
 - Reutilización de hilos para evitar el costo computacional de crear y destruir hilos constantemente.
 - Tipos de pools mediante Executors: `newFixedThreadPool`, `newCachedThreadPool`, `newSingleThreadExecutor`, etc.
- **Programación Asíncrona:**
 - `Callable<V>` **vs** `Runnable`: `Callable` permite devolver un resultado o lanzar una excepción comprobada.

- `Future<V>`: Representa el resultado futuro de una computación asíncrona.
- `CompletableFuture<V>`: Programación reactiva/asíncrona funcional que permite encadenar y combinar tareas dependientes sin bloquear hilos explícitamente.